

High-Level Synthesis for Scheduling Pipelined Circuits:

A Small List-Based Modulo Scheduler

Md Ratul Ahmed Alvi
 Department of Electrical Engineering
 Hochschule Hamm-Lippstadt (HSHL)
 Lippstadt, Germany
 md-ratul-ahmed.alvi@stud.hshl.de

Abstract—High-level synthesis translates behavioral descriptions into register-transfer level hardware. One important step is scheduling, because the selected start time of each operation determines resource use, latency, and throughput. For loop-based designs, modulo scheduling overlaps successive iterations and is mainly described by the initiation interval (II). This paper presents a small list-based modulo scheduler written in C++ and explains the resource-constrained and recurrence-constrained lower bounds on II. Three examples are studied: a 3-tap FIR filter, an accumulation loop, and a single-multiply delay line. The corrected experiments show three different situations: a resource-limited schedule, a recurrence-limited schedule, and a schedule with II equal to one. The examples also show why operation latency and resource initiation capability must be stated separately when evaluating a pipelined design.

Index Terms—high-level synthesis, modulo scheduling, list scheduling, initiation interval, pipeline, data-flow graph

I. INTRODUCTION

Designing a circuit directly in VHDL or Verilog gives the designer detailed control, but it also requires time and hardware-specific experience. High-level synthesis (HLS) reduces part of this effort by converting a behavioral description into an RTL implementation. The main HLS steps include scheduling, allocation, and binding [1]. Scheduling decides when each operation starts and therefore has a direct effect on performance and resource usage.

Loops are common in signal processing and control applications. If every iteration is executed completely before the next one starts, the resulting throughput can be low. Loop pipelining improves throughput by overlapping successive iterations. The time between the start of two iterations is called the initiation interval (II). A smaller II normally means a higher steady-state throughput.

This work studies a small list-based modulo scheduler. The purpose is not to replace a production HLS tool, but to demonstrate the main scheduling constraints in a form that can be followed and tested by a student. The paper first introduces the mathematical model, then describes the scheduler, and finally evaluates three small examples.

II. BACKGROUND

A. Data-Flow Graph

A loop body can be represented by a data-flow graph $G = (V, E)$. Each node $v \in V$ is an operation, and each edge $e = (u, v, d)$ represents a dependency from operation u to operation v . The value d is the iteration distance. An edge with $d = 0$ is inside one iteration, while $d > 0$ is a loop-carried dependency.

For a target initiation interval II , a valid modulo schedule must satisfy

$$t(v) \geq t(u) + \lambda(u) - d \cdot II, \quad (1)$$

where $t(v)$ is the start time and $\lambda(u)$ is the latency of operation u . Equation (1) is important because it handles both normal and loop-carried dependencies.

B. Resource and Recurrence Bounds

The resource-constrained initiation interval estimates how often the required operations can be started with the available hardware. If every functional unit can accept one new operation per cycle, the lower bound is

$$ResII = \max_r \left\lceil \frac{N_r}{M_r} \right\rceil, \quad (2)$$

where N_r is the number of operations of resource type r in one iteration and M_r is the number of available units. If a unit is not fully pipelined, its own initiation interval must also be included in the resource calculation.

A loop-carried cycle with total latency D and iteration distance L gives the recurrence bound

$$RecII = \left\lceil \frac{D}{L} \right\rceil. \quad (3)$$

The scheduler cannot use an II below

$$II_{\min} = \max(ResII, RecII). \quad (4)$$

These are lower bounds. A heuristic scheduler may still require a larger II if it cannot find a feasible placement.

C. Latency and Throughput

Latency is the time from the start of one iteration until its result is available. Throughput describes how frequently results are produced after the pipeline has filled. For one result per iteration,

$$\text{Throughput} = \frac{1}{II}. \quad (5)$$

A pipeline can therefore improve throughput without reducing the latency of one individual iteration.

III. LIST-BASED MODULO SCHEDULER

The implementation uses a simple iterative approach inspired by modulo scheduling methods [2], [3]. Operations are first ordered by their ASAP times. For each target II , the scheduler tries to place every operation at the earliest cycle that satisfies both dependency and modulo-resource constraints.

- 1) Compute $ResII$, $RecII$, and $II_{\min}$.
- 2) Start with $II = II_{\min}$.
- 3) Compute an ASAP-based priority order.
- 4) Place each operation at the earliest valid cycle.
- 5) Check all dependencies using Eq. (1).
- 6) If the schedule is invalid, increase II and repeat.

The resource table is indexed by $c \bmod II$, because operations from later iterations repeat in the same modulo slots. Listing 1 shows the main idea. The actual program also checks dependencies before accepting a cycle.

```

1 for (int id : order) {
2   int c = asap[id];
3
4   while (!dependenciesSatisfied(id, c, II, schedule)
5         || resourceUsed(c % II, ops[id].type)
6           >= available(ops[id].type)) {
7     ++c;
8   }
9
10  schedule[id] = c;
11  reserve(c % II, ops[id].type);
12 }

```

Listing 1. Simplified greedy placement loop.

The program represents each operation with a type, latency, dependency list, and iteration distance. The minimum information is illustrated in Listing 2.

```

1 struct Edge {
2   int source;
3   int distance;
4 };
5
6 struct Op {
7   Resource type;
8   int latency;
9   vector<Edge> deps;
10 };

```

Listing 2. Operation representation used by the scheduler.

This implementation is deliberately small. Its runtime depends on the number of operations, the number of tested II values, and how far the cycle search moves before a free valid slot is found. Therefore, a fixed $O(n \log n)$ claim would be too strong for the complete implementation. Sorting the priority list costs $O(n \log n)$, but the repeated feasibility search adds further work.

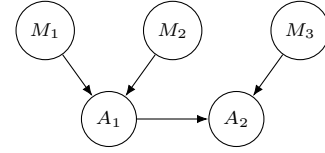


Fig. 1. Data-flow graph of the 3-tap FIR loop.

TABLE I
FIR SCHEDULE FOR ONE ITERATION, $II = 3$.

Operation	Start cycle	Resource
M_1	0	Multiplier
M_2	1	Multiplier
M_3	2	Multiplier
A_1	3	Adder
A_2	4	Adder

IV. EXPERIMENTS

All examples use fully pipelined functional units: an operation may have a latency greater than one, but the same unit can accept a new operation every cycle. This assumption is necessary to interpret Eq. (2) correctly.

A. FIR Filter: Resource-Limited Case

The first loop implements

$$y[n] = ax[n] + bx[n-1] + cx[n-2]. \quad (6)$$

It contains three multiplications and two additions. One multiplier and two adders are available. Multiplier latency is two cycles and adder latency is one cycle. The graph has no loop-carried dependency.

The resource bound is

$$ResII = \left\lceil \frac{3}{1} \right\rceil = 3, \quad RecII = 0. \quad (7)$$

The scheduler finds $II = 3$, which reaches the lower bound. Table I shows one modulo schedule. The three multiplier starts occupy the three modulo slots, while the two additions use separate adder resources.

The result latency is five cycles, while a new iteration starts every three cycles. The steady-state throughput is therefore $1/3$ result per cycle.

B. Accumulation: Recurrence-Limited Case

The second loop is

$$sum += x[i] \cdot coeff. \quad (8)$$

One multiplier and one adder are available. Both units are fully pipelined. Multiplier latency is two cycles. To create a clear recurrence-limited example, the accumulator addition is modeled with latency two cycles. Its result is used by the next iteration, giving a recurrence with $D = 2$ and $L = 1$.

The bounds are

$$ResII = 1, \quad RecII = \left\lceil \frac{2}{1} \right\rceil = 2. \quad (9)$$

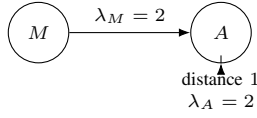


Fig. 2. Accumulation loop with a loop-carried recurrence.

TABLE II
ACCUMULATION SCHEDULE, $II = 2$.

Iteration	M start	A start	Result
0	0	2	4
1	2	4	6
2	4	6	8
3	6	8	10

The scheduler therefore starts at $II_{\min} = 2$ and finds a feasible schedule. Table II shows the repeated start pattern.

This is a genuine recurrence-limited result. Adding more multipliers or adders does not reduce II below two unless the recurrence is restructured or its latency is reduced.

C. Delay Line: II Equal to One

The third loop computes

$$y[n] = x[n] \cdot \text{coeff}. \quad (10)$$

It contains one multiplication per iteration, two multipliers are available, and there is no loop-carried dependency. Thus,

$$\text{ResII} = 1, \quad \text{RecII} = 0, \quad II = 1. \quad (11)$$

The multiplier latency remains two cycles, but a new iteration begins every cycle. After the pipeline is filled, one result is produced per cycle.

D. Summary

Table IV compares the three cases. The results match the intended lower bound in each example.

E. Validation Method

Each experiment was checked in three steps. First, the analytical lower bounds were calculated using Eqs. (2)–(4). Second, the C++ scheduler generated start times for all operations. Third, an independent validation routine checked every dependency and every modulo-resource slot.

For each edge (u, v, d) , the validator evaluates Eq. (1). For each resource type, it also counts the operation starts mapped to every $c \bmod II$ slot. A schedule is accepted only when all dependency inequalities hold and no slot uses more units than are available. These checks are small, but they prevent the scheduler from reporting a result that only appears feasible.

The experiments were deliberately chosen so that one expected limit was clear in each case. This made it possible to compare the observed II with a known analytical expectation instead of reporting the program output without a reference value.

TABLE III
DELAY-LINE SCHEDULE, $II = 1$.

Iteration	Multiply start	Result cycle
0	0	2
1	1	3
2	2	4
3	3	5
4	4	6

TABLE IV
SUMMARY OF CORRECTED EXPERIMENTS.

Example	ResII	RecII	II
3-tap FIR	3	0	3
Accumulation	1	2	2
Delay line	1	0	1

V. HARDWARE/SOFTWARE CO-DESIGN VIEW

The experiment can also be understood as a small hardware/software co-design workflow. The scheduling algorithm runs as software and receives a behavioral model, operation latencies, dependency information, and resource limits. Its output is a cycle-level schedule. That schedule then guides the hardware architecture by showing when multipliers and adders are required and which operations may share the same unit.

The software side performs design-space reasoning: it tests the lower bounds, searches for valid start times, and reports whether the current resource budget is sufficient. The hardware side is represented by the selected number of functional units and their pipeline behavior. For example, the FIR result shows that one multiplier forces three multiplication start slots per iteration. This information can be used to decide whether the extra area of a second multiplier is justified by a lower II .

The separation is useful because hardware changes can be evaluated first at the scheduling level. However, the schedule is still only a model. A complete flow would also generate RTL, synthesize it for an FPGA, and compare the predicted II with simulation and timing reports.

VI. DISCUSSION

The examples show why the dominant bound must be identified before changing the hardware. In the FIR filter, the resource bound dominates. Adding a second multiplier could lower ResII , although a new schedule would still be required. In the accumulation loop, the recurrence dominates. Additional units alone do not change the two-cycle feedback path. In the delay line, neither bound is greater than one.

The corrected accumulation example also shows an important modeling lesson. An ordinary dependency from a multiplication to an addition inside the same iteration does not by itself create RecII . A recurrence bound requires a loop-carried dependency. The latency around that loop-carried cycle and its iteration distance determine Eq. (3).

The implementation remains a teaching model. Production modulo schedulers handle memory dependencies, control flow,

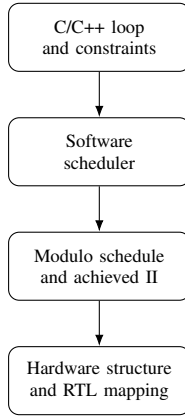


Fig. 3. Simplified hardware/software co-design flow used in this work.

chaining, operator-specific initiation intervals, and schedule-length optimization. Exact formulations can prove optimal IIs for many practical loops, but they are more complex than the list-based approach used here [4]. Heuristic and SDC-based approaches offer other trade-offs between scheduling quality and runtime [3].

VII. CONCLUSION

This paper presented a small list-based modulo scheduler for pipelined loops in HLS. The scheduler starts from the

maximum of the resource and recurrence bounds and increases II only when a feasible schedule cannot be found.

Three corrected examples demonstrated different limits. The FIR filter reached $II = 3$ because one multiplier had to start three multiplications per iteration. The accumulation loop reached $II = 2$ because of a two-cycle loop-carried recurrence. The delay line reached $II = 1$ because neither resources nor dependencies required a larger interval. The main result is that operation latency, functional-unit initiation capability, and loop-carried dependence must be modeled separately. Without this distinction, an observed II can easily be attributed to the wrong cause.

REFERENCES

- [1] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.
- [2] B. R. Rau, "Iterative modulo scheduling: An algorithm for software-pipelining loops," in *Proc. 27th Annu. Int. Symp. Microarchitecture*, 1994, pp. 63–74.
- [3] Z. Zhang and B. Liu, "SDC-based modulo scheduling for pipeline synthesis," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design*, 2013, pp. 211–218.
- [4] J. Oppermann, M. Reuter-Oppermann, L. Sommer, A. Koch, and O. Sinnen, "Exact and practical modulo scheduling for high-level synthesis," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 2, pp. 1–26, 2019.